Week 4 Lecture Notes: Design Patterns for ML

Strategy for models, Observer for training callbacks, Factory for dataset loaders, pytest

AI for Aerospace & Mechanical Engineering (PhD), Block 0

> **Provenance:** new content. The instructor's PDFs do not cover software design patterns. The pattern material is rewritten (not copied) from GeeksforGeeks ("Strategy Design Pattern", "Observer Design Pattern", "Factory Method - Python Design Patterns", "Getting Started with Pytest", "Pytest Fixtures") and E. Gamma, R. Helm, R. Johnson, J. Vlissides, *Design Patterns* (1994); Python details follow W3Schools (Classes, Inheritance, Polymorphism, Dictionaries, Lambda). Persian terminology was checked against the KodAmoz Python bootcamp (kodamoz.com/fa/bootcamps/python), which covers OOP but not patterns or testing. Least squares and k-NN are adapted from C. M. Bishop, *Pattern Recognition and Machine Learning*, Ch. 3 and Sec. 2.5.2; early stopping from Goodfellow, Bengio and Courville, *Deep Learning*, Sec. 7.8; the fuel-flow model from J. D. Anderson, *Aircraft Performance and Design*, Ch. 5.

1. The problem

We want to predict the fuel flow $\dot{m}_f$ [g/h] of a small piston UAV from airspeed V, altitude h, climb rate $\dot{h}$ and mass m. Week 6 treats this as a regression problem; this week builds the software that Weeks 5 to 13 reuse. The usual first attempt is one function, `run_experiment(model_type, data_source, verbose, early_stop, patience)`, with an `if/elif` chain for the model, another for the data source, and the logging and stopping logic inside the training loop. It fails in three ways, all shown live in the lab. A misspelled `"KNN"` falls through to an `else` that returns the mean baseline **without any error**. Its k-NN branch does not scale the features, a bug nobody sees because nothing compares it with anything. And every new model, data source or training feature edits the same function, so nothing can be tested in isolation.

The function changes along three independent **axes**: the model, what happens during training, and where the data comes from. Three classic design patterns isolate one axis each.

What varies	Pattern	Course classes (`src/aerospace_ai/`)
the model	Strategy (+ Adapter)	`models.Regressor`, `models.RegressionExperiment`
training behaviour	Observer	`training.Trainer`, `training.Callback`
the data source	Factory	`datasets.DatasetFactory`, `datasets.DatasetLoader`

The goal in all three is the **Open/Closed Principle**: code should be open for extension and closed for modification. Adding a model, a callback or a loader must mean adding a class, not editing code that already works.

Worked Example 1: fuel flow at 25 m/s

Power required is parasite plus induced plus climb power; shaft power is P_{req}/η_p; fuel flow is BSFC times shaft power:

$$P_{\text{req}} = \tfrac{1}{2}\rho V^3 S C_{D0} + \frac{2W^2}{\rho V S \pi e\, AR} + W\dot{h}, \qquad \dot{m}_f = \text{BSFC} \cdot \frac{P_{\text{req}}}{\eta_p}.$$

With $\rho = 1.225$ kg/m³, $V = 25$ m/s, $S = 1.5$ m², $C_{D0} = 0.03$, $m = 30$ kg ($W = 294.20$ N), $AR = 10$, $e = 0.8$, $\eta_p = 0.75$ and BSFC $= 0.35$ kg/kWh, level flight: $P_{\text{par}} = 0.5 \times 1.225 \times 15\,625 \times 1.5 \times 0.03 = 430.66$ W, $P_{\text{ind}} = 173\,106/1154.5 = 149.94$ W, shaft power $580.60/0.75 = 774.13$ W, and $\dot{m}_f = 0.35 \times 0.77413 = 0.27095$ kg/h $= 270.95$ g/h. A 1 m/s climb adds $W\dot{h}/\eta_p = 392.3$ W and gives 408.24 g/h. The model is convex in V and contains the product $m\dot{h}$, so no straight line fits it exactly. The 400-row course dataset (`fuel_flow_synthetic`, seed 4) adds 3 % measurement noise.

2. Strategy: the model as a replaceable part

The Strategy pattern puts each algorithm in its own class behind a common interface. The **context**, the code that uses the algorithm, holds a reference to "some strategy" and calls only the interface; it never asks which algorithm it holds. Swapping is one assignment, and a new algorithm is one new class.

In Python the lightest strategy is a function: functions are objects, so a dictionary `{"mean": lambda ..., "median": lambda ...}` is already a strategy table, and an unknown key raises `KeyError` instead of falling through. Functions suffice while a strategy has no state and no settings. Models have both (learned `coef_`, chosen `k` or `alpha`) and need the same input checks every time, so the course uses an abstract base class. `Regressor(ABC)` has public `fit` and `predict` methods that check shapes, finite values and the fit status once, then call the abstract hooks `_fit` and `_predict`. These are template methods, as `BaseSensor.add_reading` was in Week 2. The concrete strategies are `MeanBaseline`, `LinearLeastSquares` (normal equations, optional ridge penalty), `KNearestNeighbors` (with optional standardisation) and `GradientDescentRegressor`. The context `RegressionExperiment` fits a strategy on a training set, scores it on a test set and returns a frozen `EvaluationResult`; `compare` runs a list of strategies on the same split.

An **Adapter** converts one interface into another. scikit-learn estimators have `fit` and `predict` but are not `Regressor` subclasses; `SklearnRegressor(estimator)` wraps any object whose *shape* matches the `SupportsFitPredict` protocol. A `typing.Protocol` with `@runtime_checkable` checks the methods an object has, not its ancestry: duck typing with type hints.

Worked Example 2: three strategies on four flights

Training points $(V, \dot{m}_f)$: $(18, 170)$, $(22, 200)$, $(26, 280)$, $(30, 400)$; test points $(25, 270)$ and $(29, 375)$.

- Mean: $\bar{y} = 262.5$; errors -7.5 and -112.5; RMSE $= \sqrt{(56.25 + 12\,656.25)/2} = 79.73$ g/h.
- k-NN, $k = 2$, no scaling: the neighbours of 25 are 26 and 22, giving 240; of 29 are 30 and 26, giving 340. RMSE $= \sqrt{(900 + 1225)/2} = 32.60$ g/h.
- Least squares: $\bar{x} = 24$, $S_{xy} = \sum (x_i - \bar{x})(y_i - \bar{y}) = 1540$, $S_{xx} = 80$, slope 19.25, intercept $262.5 - 19.25 \times 24 = -199.5$; predictions 281.75 and 358.75; RMSE $= 14.18$ g/h.

The line wins inside the training range, but at $V = 16$ m/s (true value about 162 g/h) it predicts 108.5 while k-NN predicts 185. The best strategy depends on the question, which is exactly why swapping must be cheap.

On the full dataset with a chronological 300/100 split, the linear models (OLS, ridge, gradient descent) reach a test RMSE of 51.1 to 51.5 g/h, scaled k-NN ($k = 5$) 51.6 g/h, a random forest through the adapter 37.65 g/h, and the mean baseline 225.4 g/h. Unscaled k-NN reaches 235.3 g/h, **worse than the mean**: altitude (0 to 3000 m) swamps airspeed (16 to 36 m/s) in the distance. Only `experiment.strategy` changed between runs, and the comparison exposed the scaling bug.

3. Observer: training events without touching the loop

A training loop has many interested parties: a history recorder, a logger, an early-stopping rule, a divergence guard, a learning-rate schedule, later a checkpoint writer. The Observer pattern lets a **subject** keep a list of **observers** and notify all of them when something happens, without knowing what they do. In the course library the subject is `Trainer` and the observers are `Callback` objects with three hooks: `on_train_begin`, `on_epoch_end` and `on_train_end`. The hooks are deliberately not abstract, because each observer cares about only some events. Observers communicate only through the shared `TrainingState` dataclass (epoch, losses, learning rate, `stop_training`, `stop_reason`); they never call each other. `Trainer.fit` initialises the model, notifies `on_train_begin`, then for each epoch takes one `train_step`, computes the validation loss, notifies `on_epoch_end` and breaks if a callback requested a stop, and finally notifies `on_train_end`. The model only has to satisfy the `TrainableModel` protocol (`initialize`, `train_step`, `loss`, `get_state`, `set_state`), so the same loop will train PyTorch models in Block 2.

Worked Example 3: one gradient-descent step

For $\hat{y} = w_0 + w_1 x$, data $(1, 2), (2, 4), (3, 5)$, $w = (0, 0)$ and MSE loss: the residuals are $(-2, -4, -5)$ and $L_0 = 15$. With rows $A_i = (1, x_i)$ the gradient is $\nabla L = \frac{2}{n} A^\top (Aw - y) = (-7.333, -16.667)$. A step with $\eta = 0.1$ gives $w = (0.7333, 1.6667)$, predictions $(2.4, 4.0667, 5.7333)$ and $L_1 = 0.2341$. With $\eta = 0.5$ it gives $w = (3.667, 8.333)$ and $L_1 = 309$: the step overshoots. For a quadratic loss gradient descent is stable only if $\eta < 2/\lambda_{\max}(H)$ with $H = \frac{2}{n} A^\top A$; here $\lambda_{\max} = 11.09$, so $\eta < 0.18$. On the standardised fuel data $\lambda_{\max} = 2.17$, which is why $\eta = 0.1$ works and $\eta = 1.5$ diverges. `DivergenceGuard` stops a run whose loss becomes non-finite or exceeds ten times the first epoch's loss.

Worked Example 4: early stopping by hand

`EarlyStopping(patience=2, min_delta=0.05)` sees validation losses 1.00, 0.80, 0.78, 0.70, 0.69, 0.68, 0.50. An epoch improves only if its loss is below best -0.05. Epochs 0, 1 and 3 improve; epoch 2 (0.78 is not below 0.75) does not. The counter reads 0, 0, 1, 0, 1, 2, so training stops after epoch 5 and the weights of epoch 3 (0.70) are restored. With patience 3 the run would have reached epoch 6 and the much better 0.50: patience trades compute for the risk of quitting on a plateau. On the fuel data, `min_delta = 1` $(\text{g/h})^2$ stops after 39 epochs instead of 170, with the same test RMSE (51.19 vs 51.21 g/h).

Two practical rules. Observers run in **attachment order**: a logger attached before `StepDecay` prints the old learning rate, after it the new one, although training is identical because the Trainer reads the rate at the start of the next epoch. And observers must reset their counters in `on_train_begin`, so that one object can serve several runs.

4. Factory: ask for data by name

Data sources are the third axis: today a synthetic generator and a CSV file, next week a noisy LiDAR log, in Week 11 a folder of camera images. The Factory pattern moves object creation out of the client: the client names what it wants and depends only on the abstract product, `DatasetLoader`, whose single method `load()` returns a `Dataset` (a frozen dataclass with `X`, `y`, names and a leak-free chronological `split`). The Pythonic factory is a **registry**: a class-level dictionary `_registry: ClassVar[dict[str, type[DatasetLoader]]]` filled by the class decorator `@DatasetFactory.register("name")`. A decorator is just a function that receives the class, stores it and returns it unchanged. `DatasetFactory.create(name, **kwargs)` instantiates the loader with its settings; `load(name, **kwargs)` also calls `load()`. Duplicate names and non-loader classes are rejected, and an unknown name raises `UnknownDatasetError` with the closest registered names from `difflib`. The library ships `csv` (any table), `flight_log_csv` (the Week 1 log with a derived load-factor column, reusing `FlightLog`) and `fuel_flow_synthetic`. A new source is one decorated class; in the lab, `fuel_flow_csv` takes six lines.

5. Testing patterns with pytest

Patterns make testing easier because each piece talks only to an interface. Three techniques are used in `tests/test_models.py`, `test_training.py` and `test_datasets.py` (96 tests, about 2 s).

- **Contract tests.** A fixture with `params=` yields every strategy, and the same six tests run on each: `fit` returns `self`, predictions have the right shape, `predict` before `fit` raises `NotFittedError`, wrong feature counts and bad data raise `ValueError`, and every model beats or equals the mean baseline. A new model added to the list is tested for free. This is the Liskov Substitution Principle as code: any strategy must work where a `Regressor` is expected.
- **Test doubles.** A `SpyCallback` records the events it receives, so one assert checks the exact notification order. A `ScriptedModel` returns a fixed list of validation losses, which turns Worked Example 4 into a test. A `FakeEstimator` tests the adapter without scikit-learn.
- **Isolation.** A fixture registers a throw-away loader and unregisters it after the test (code after `yield` is the teardown); `tmp_path` provides a fresh folder for CSV files, `capsys` captures printed output, `pytest.approx` compares floats, and `pytest.importorskip` skips a test when an optional library is missing.

6. Summary

Strategy isolates *what* algorithm runs, Observer *what else happens* while it runs, and Factory *where its inputs come from*. Each pattern costs a few classes and pays back with code that grows by addition. Practice 4 extends all three without editing the library: a polynomial strategy with interaction features (test RMSE 22.59 g/h instead of 51.46, because climb power is $\dot{Wh}$), a `ReduceLROnPlateau` callback, and a loader that filters any registered dataset by flight regime.